

Concurrency & Distributed Systems

Week 1 — Part 3: Theoretical Framing (Why Concurrency Works and Fails)

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

Why Concurrency Works (and Fails)

Practical systems fail for mathematical reasons.

Before adding threads blindly, we must understand three principles:

- **Amdahl's Law** (limits of speedup)
- **Little's Law** (queue behavior)
- **Context switching cost** (hidden overhead)

Senior-level mindset

You must justify performance claims with math **and** measurements.

Amdahl's Law — The Ceiling

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- P = parallelizable fraction
- N = number of cores/threads
- $S(N)$ = maximum theoretical speedup

Interpretation

The sequential part $(1 - P)$ dominates as N grows.

Worked Example

Assume:

- $P = 0.8$ (80% parallelizable)
- $N = 10$

$$S(10) = \frac{1}{0.2 + \frac{0.8}{10}} = \frac{1}{0.2 + 0.08} = \frac{1}{0.28} \approx 3.57$$

Key point

Even 10 threads cannot give $10\times$ speedup.

Infinite Cores Limit

As $N \rightarrow \infty$:

$$S_{\max} = \frac{1}{1 - P}$$

If $P = 0.8$:

$$S_{\max} = \frac{1}{0.2} = 5$$

Meaning

Even infinite cores cannot exceed $5\times$ speedup.

Student Calculation (1 minute)

Question: If $P = 0.5$, what is the maximum speedup with infinite cores?

$$S_{\max} = \frac{1}{1 - 0.5} = 2$$

Lesson

If half the program is sequential, scaling is fundamentally limited.

Student Instructions: Run AmdahlCalculator (Mandatory)

Step 1 — Compile

```
javac AmdahlCalculator.java
```

Step 2 — Run

```
java AmdahlCalculator
```

Step 3 — Record

- Copy the printed table into your report (or take a screenshot)
- Highlight the “inf cores” column

AmdahlCalculator.java (Core Function)

Focus on the formula:

```
public static double calculateSpeedup(  
    double P, int N) {  
    return 1.0 / ((1.0 - P) + (P / N));  
}
```

What to watch

As N grows, the speedup increases then **plateaus**.

Expected Pattern (Sanity Check)

Your exact rounding may differ, but the **limits** must match:

- $P = 0.50 \Rightarrow S_{\max} \approx 2.00$
- $P = 0.75 \Rightarrow S_{\max} \approx 4.00$
- $P = 0.90 \Rightarrow S_{\max} \approx 10.00$
- $P = 0.95 \Rightarrow S_{\max} \approx 20.00$

If your output does not match

Your formula implementation is wrong.

Little's Law — Why Queues Form

$$L = \lambda W$$

Where:

- L = average number of requests in the system
- λ = arrival rate (req/s)
- W = average time in system (seconds)

Key idea

If latency W increases, queue size L increases automatically.

Little's Law Example

If:

- $\lambda = 50$ req/s
- $W = 0.2$ s (200ms)

$$L = 50 \times 0.2 = 10$$

Interpretation

On average, 10 requests are inside the system at once.

Student Exercise (1 minute)

Compute L if:

$$\lambda = 80 \text{ req/s}, \quad W = 0.25 \text{ s}$$

$$L = 80 \times 0.25 = 20$$

Meaning

Your system must handle about 20 concurrent requests on average.

Context Switching Cost — Hidden Overhead

Threads are not free. Context switching can imply:

- saving/restoring registers
- pipeline disruption
- cache pollution
- scheduler overhead

When thread count is huge

The OS scheduler thrashes and p95 latency explodes.

Context Switching Example

Assume:

- switch cost = $5\ \mu s$
- 100,000 switches/sec

$$100,000 \times 5\mu s = 0.5s$$

Interpretation

Half the CPU time is wasted switching.

- **Amdahl:** parallelism has a ceiling.
- **Little:** latency creates queues.
- **Context switching:** too many threads hurt.

Balance

Idle time exploitation vs scheduling overhead.

Transition Activity: Predict – Then – Test

Scenario: “We will add threads next.”

Predict what happens to:

- ① Latency (p50)
- ② Throughput
- ③ CPU usage

Write predictions now. We test in Part 2 experiments.

Common Misconceptions Quiz (Pair Defense)

True/False (defend your answer in pairs):

- ❶ High throughput always means low latency. (False)
- ❷ Adding more threads always improves performance. (False)
- ❸ Parallelism requires multiple cores; concurrency does not. (True)
- ❹ If CPU is 10%, the system is underutilized. (Depends)
- ❺ Concurrency is only useful for I/O-bound tasks. (Mostly true; nuanced)

Students repeat back:

- Concurrency $\neq$ speed; concurrency hides waiting
- Latency $\neq$ throughput (both matter)
- Most servers are I/O-bound

Homework (mandatory):

- Run AmdahlCalculator
- Paste output table (or screenshot) into your report
- Answer: “Why does speedup plateau?”

Exit Ticket (2 minutes)

Answer quickly:

- 1 Define latency in one sentence.
- 2 Why can CPU be low while latency is high?
- 3 True/False: more threads always help. (one keyword)

Key takeaway

You cannot parallelize waiting away. You must hide it.